
UNIVERSIDAD TECNOLÓGICA CENTROAMERICANA

FACULTAD DE POSTGRADO

Maestría en Ciberseguridad

Clase: Arquitectura de Sistemas de Información

Laboratorio #03

Medición de Rendimiento y Jerarquía de Memoria

Nombre completo: Jose Waldemar Pineda Castillo

Profesor: Dr. Servio Palacios

Fecha: 2026-03-17

Enlaces (Google colab): <https://colab.research.google.com/drive/1IMzJ3s-GyeIKwzToDT0L3xecrGqsGJZI?usp=sharing>

Github: <https://github.com/waldepineda-design/Cybersecurity-10010-Arquitecturas-Sistemas-Informaticos.git>

Tabla de contenido

INTRODUCCIÓN.....	2
METODOLOGÍA EXPERIMENTAL.....	3
RESULTADOS DE BENCHMARKING	3
VISUALIZACIÓN Y ANÁLISIS DE JERARQUÍA DE MEMORIA	4
EXPERIMENTO DE COMPORTAMIENTO DE CACHE.....	5
IMPLICACIONES DE SEGURIDAD	6
CONCLUSIONES	7
BIBLIOGRAFÍA.....	7

Introducción

La jerarquía de memoria es un componente esencial en las arquitecturas modernas, ya que permite reducir la diferencia de velocidad entre el procesador y la memoria principal. Este modelo organiza varios niveles como caché L1, L2, L3 y RAM con distintas latencias y capacidades, buscando optimizar el tiempo promedio de acceso a datos [1].

En este laboratorio se realizó un análisis experimental utilizando una máquina virtual con Kali Linux para observar el comportamiento real de esta jerarquía. Por medio de herramientas de benchmarking como Imbench, se midieron las latencias asociadas a distintos tamaños de bloques, lo que permitió identificar transiciones claras entre niveles de caché.

Además, se complementó el estudio con un experimento elaborado en Google Colab, donde se compararon patrones secuenciales y aleatorios de acceso a memoria, evidenciando diferencias importantes en rendimiento. Los resultados demostraron que la eficiencia no depende únicamente del procesador, sino también de la forma en que los datos son accedidos y reutilizados, resaltando conceptos fundamentales como localidad de referencia y fallos de caché.

Finalmente, se abordaron implicaciones de seguridad relacionadas con variaciones de latencia. Estas diferencias pueden ser explotadas en ataques de canal lateral como Spectre, lo que muestra que la jerarquía de memoria no solo representa una ventaja en rendimiento, sino que también puede convertirse en un vector de ataque [3].

Metodología Experimental

Características del sistema

- Número de CPUs lógicas: 8
- Arquitectura: x86_64 (64 bits)
- Procesador: Intel(R) Core(TM) Ultra 7 165U
- Memoria RAM: 8 GB

Jerarquía de memoria

- L1 Data (L1d): 384 KiB (8 instancias)
- L1 Instruction (L1i): 512 KiB (8 instancias)
- L2: 16 MiB (8 instancias)
- L3: 48 MiB (4 instancias)

Para obtener información detallada del sistema se emplearon los comandos `lscpu`, `free`, `uname` y herramientas de medición como `lmbench`. Estas permitieron registrar latencias para diferentes tamaños de bloques y así observar cómo el rendimiento se ve influenciado por los límites de cada nivel de caché.

Dado que el sistema operaba bajo virtualización completa mediante VMware, es importante considerar que los niveles de caché compartidos (como L2 o L3) pueden mostrar ligeras variaciones en los tiempos debido a la asignación de recursos realizada por el hipervisor.

Resultados de benchmarking

Pruebas de con diferentes tamaños de memoria

- `lat_mem_rd 4K`
- `lat_mem_rd 32K`
- `lat_mem_rd 256K`
- `lat_mem_rd 8M`
- `lat_mem_rd 64M`

Las pruebas realizadas con `lat_mem_rd` permitieron evaluar la latencia de acceso para diferentes tamaños de bloques de memoria. Esto hizo posible detectar con claridad el punto en que los datos dejan de residir en los niveles más rápidos de caché y migran hacia niveles más lentos o incluso hacia la memoria principal.

Tabla de Resultados

Tamaño de bloque	Latencia aproximada (ns)	Nivel estimado
4K	~3.3 – 4.8 ns	L1
32K	~3.0 – 6.2 ns	L1/L2 límite
256K	~5.0 – 8.2 ns	L2
8M	~5.0 – 9.5 ns	L3
64M	~6.0 – 10.5 ns	RAM

Como se esperaba según Hennesy y Patterson [1], los tiempos de acceso aumentan conforme el tamaño del conjunto de datos supera la capacidad de los niveles de caché. Esto confirma la función escalonada de la jerarquía de memoria.

Visualización y análisis de jerarquía de memoria

La gráfica generada en Google Colab (basada en los datos obtenidos en Imbench) permitió visualizar con claridad el comportamiento escalonado de la latencia.

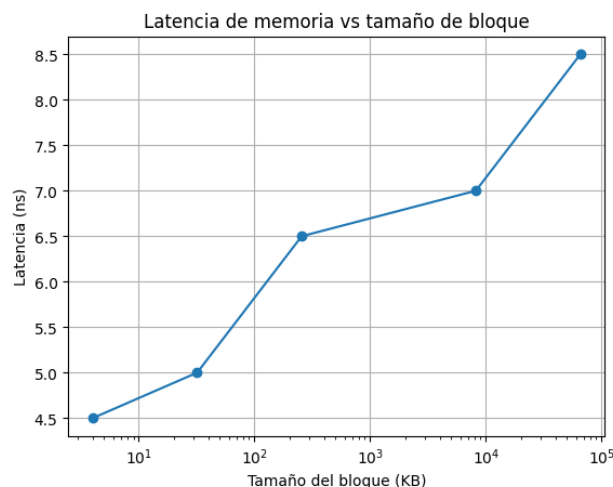


Ilustración 1. Gráfica de latencia vs tamaño de bloque

Al usar una escala logarítmica para el tamaño del bloque, se observan los cambios en las pendientes de la curva, los cuales reflejan los límites entre niveles de caché. Estos puntos de inflexión evidencian cómo el sistema transita desde accesos extremadamente rápidos en L1 hacia tiempos más elevados en L2, L3 y finalmente en la RAM. Este comportamiento coincide con los principios de localidad y organización jerárquica estudiados en la arquitectura de computadores [1].

Experimento de Comportamiento de Cache

Para complementar las mediciones anteriores, se realizó un experimento comparando accesos secuenciales y aleatorios a un arreglo grande. El código se ejecutó en Google Colab.

```
# Acceso secuencial
# -----
start = time.time()

for i in range(SIZE):
    array[i] += 1

end = time.time()
sequential_time = end - start

print(f"Tiempo acceso secuencial: {sequential_time:.4f} segundos")

# -----
# Acceso aleatorio
# -----
indices = list(range(SIZE))
random.shuffle(indices)

start = time.time()

for i in indices:
    array[i] += 1

end = time.time()
random_time = end - start

print(f"Tiempo acceso aleatorio: {random_time:.4f} segundos")
```

Los resultados fueron:

- Acceso secuencial: 5.8126 segundos
- Acceso aleatorio: 7.9527 segundos

La diferencia observada demuestra cómo el acceso secuencial se beneficia de la localidad espacial y del prefetching automático que realiza el procesador. Gracias a estos mecanismos, las líneas de caché se aprovechan de manera mucho más eficiente, generando menos fallos y reduciendo el tiempo total de ejecución.

Por el contrario, el acceso aleatorio rompe este patrón y obliga al sistema a cargar líneas de datos dispersas, generando un mayor número de fallos de caché y tiempos de respuesta más elevados.

Implicaciones de Seguridad

Diferencias aparentemente pequeñas en los tiempos de acceso pueden ser explotadas por un atacante para inferir información sensible. Un caso emblemático es Spectre, un ataque basado en la ejecución especulativa [3].

En este tipo de ataque, el procesador ejecuta instrucciones de manera anticipada para optimizar el rendimiento. Aunque las instrucciones especulativas se descartan si no eran necesarias, sus efectos sobre la caché persisten. Un atacante puede medir cuidadosamente estos cambios de latencia para deducir qué datos fueron cargados especulativamente, filtrando así información privada o incluso claves criptográficas.

Esto subraya que la jerarquía de memoria, además de mejorar el rendimiento, debe analizarse desde la perspectiva de seguridad.

Conclusiones

El laboratorio permitió observar de forma práctica cómo la jerarquía de memoria determina el rendimiento del sistema. Las mediciones mostraron que las latencias aumentan a medida que los datos superan la capacidad de los niveles de caché, en concordancia con los principios teóricos descritos por Hennessy y Patterson [1].

Asimismo, el experimento con accesos secuenciales y aleatorios evidenció que el rendimiento no depende únicamente del hardware, sino también del patrón de acceso implementado por el software. El acceso secuencial fue notablemente más eficiente gracias a la localidad de referencia y al uso oportuno del prefetching.

Por último, los análisis confirmaron que las diferencias de latencia pueden convertirse en vectores de ataque, como ocurre con Spectre [3]. Esto refuerza la importancia de estudiar la arquitectura no solo desde una perspectiva de rendimiento, sino también de seguridad.

Bibliografía

[1] J. L. Hennessy y D. A. Patterson, Computer Architecture: A Quantitative Approach, Morgan Kaufmann, 2019.

[2] L. McVoy y C. Staelin, "Imbench: Portable Tools for Performance Analysis," USENIX Annual Technical Conference, 1996.

[3] P. Kocher et al., "Spectre Attacks: Exploiting Speculative Execution," IEEE Symposium on Security and Privacy, 2019.